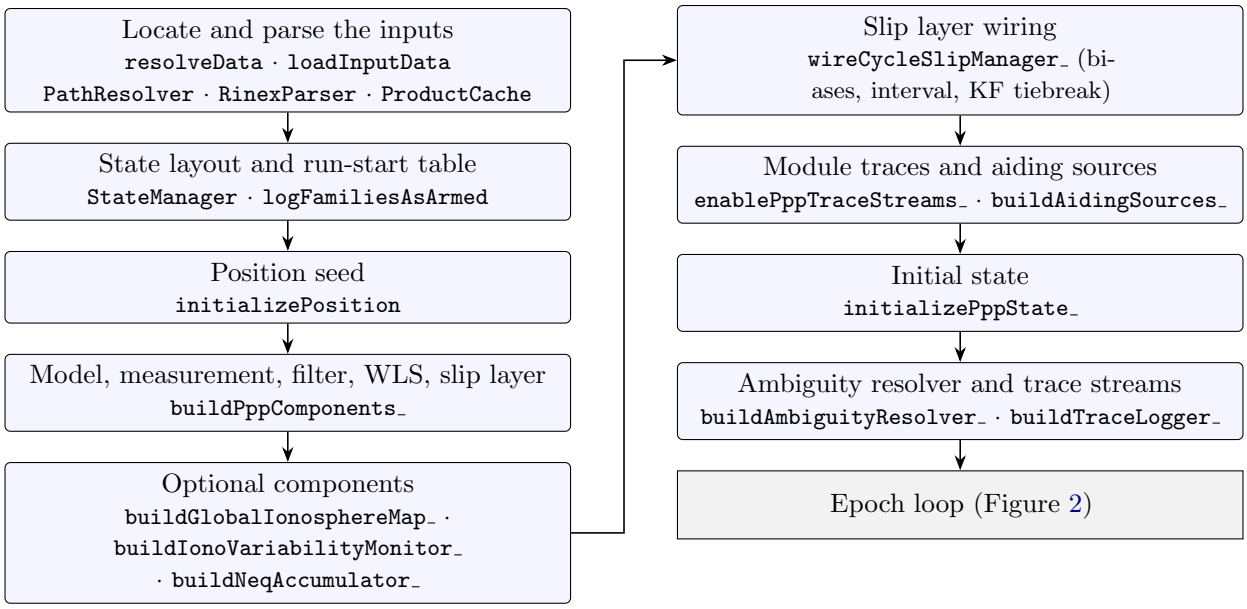


Figure 1: Preparation of a PPP or PPP-RTK run in `Application::runPpp`. The steps after `initializePosition` are private members defined in `app/PppSetup.cpp`, called in this order. `StateManager` declares the state families once and prints them as the run-start table *state families as armed*; the optional components are built only when their settings are on.

1 Pipeline and architecture diagrams

This section shows the control and data flow behind the algebra of the other chapters. For the single-receiver path it gives the preparation of a run (Figure 1), the per-epoch loop (Figure 2) and the shared geometry kernel behind the three differencing regimes (Figure 3). For the real-time path it gives the order of the seams of one epoch, with the pre-update innovation screen (Figure 4), the SSR correction pipeline (Figure 5) and the streaming front end that feeds it (Figure 6). For the network estimator it gives the joint estimator (Figure 7), the structure of a run (Figure 8), the epoch before and after the solve (Figures 9 and 10), the fourteen stages of the solve (Figure 11), the emission (Figure 12) and the harvest of the carried state (Figure 13). Each box names the function or class that implements the step, so the diagrams also serve as a map into `src/`. The diagrams are maintained by hand and simplify the code: they show the main sequence, not every gate or diagnostic.

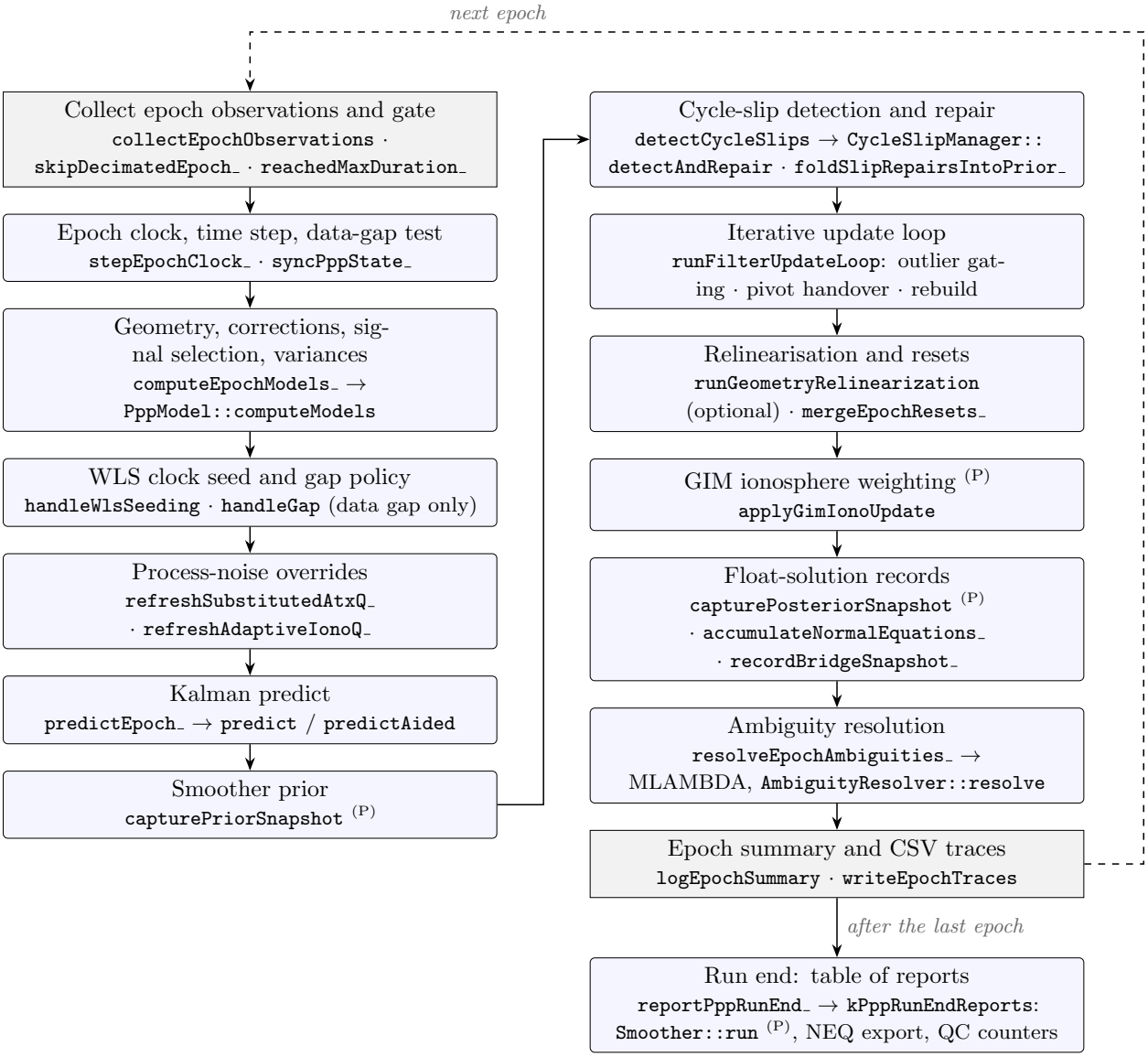


Figure 2: Per-epoch loop of `Application::runPpp`. Solid arrows give the sequence within an epoch. The first column runs from the observations to the prediction; the second column runs from the slip layer to the output, after which the loop returns to the first box for the next epoch. Steps tagged (P) run in post-processing only; the real-time path (`RealTimeEngine`, Figure 4) has no GIM source and no smoother. The smoother runs once, after the forward pass, as one report of the run-end table, and only when `processing.run_smoother` is set. The values the stages hand to one another are the members of one `PppEpochWork`, built empty at the top of each epoch.

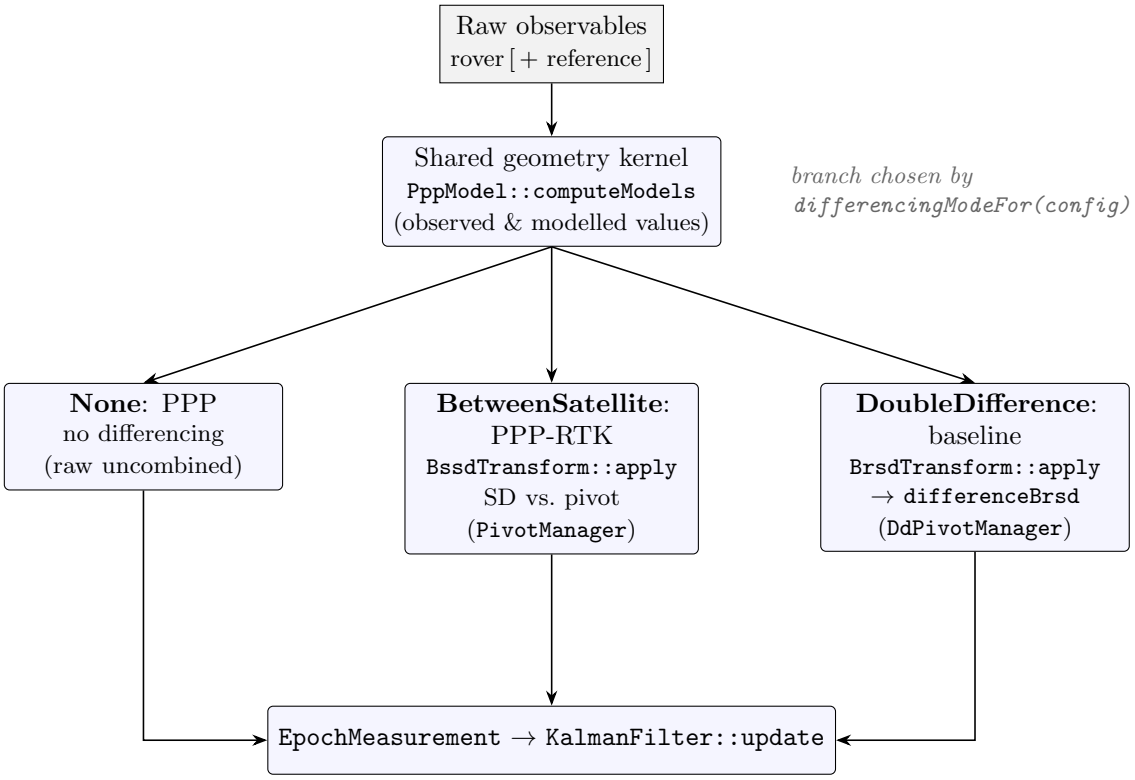


Figure 3: The three differencing regimes share one geometry kernel. The undifferenced observed and modelled values are computed once. The regime returned by `differencingModeFor` (selected once, when `MeasurementBuilderFactory` builds the measurement builder) then applies no transform (PPP), a between-satellite single difference against a pivot (PPP-RTK), or a between-receiver → between-satellite double difference (baseline). All three feed the same Kalman update.

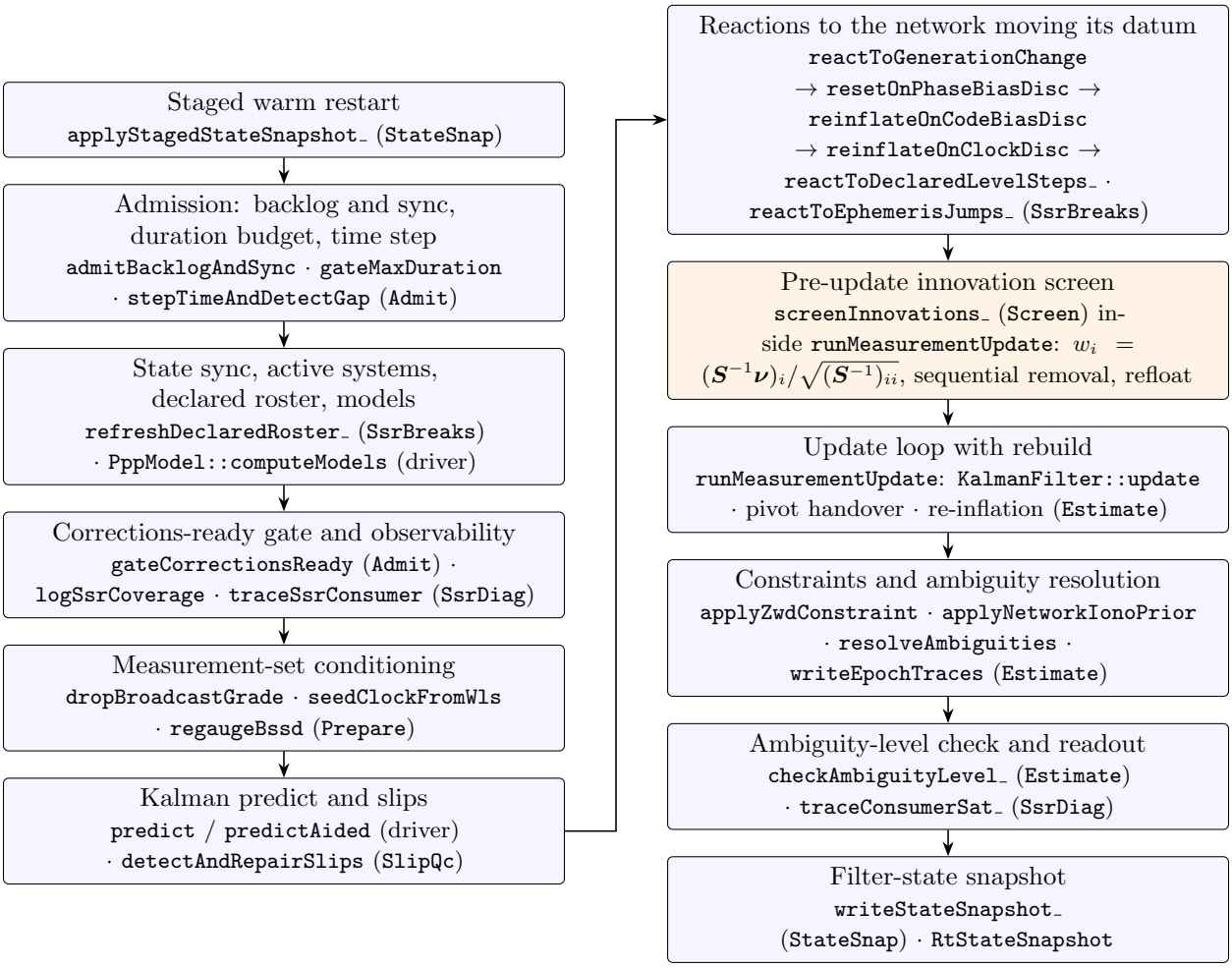


Figure 4: Order of the seams of `RealTimeEngine::processEpoch`. The names in parentheses are the files `RealTimeEngine<name>.cpp` that hold each seam. Only the admission gates, the corrections-ready gate and the empty-epoch checks of the driver can end an epoch; everything after them is straight-line. The reaction to the generation change comes first because it subsumes the finer reactions that follow. The shaded box is the user's pre-update innovation screen (the innovation-screen section): it tests every row against the predicted state before the first update pass can spread a gross row over the states, and builds that first pass itself with the flagged rows held out. The other seams with a section of their own are the staged warm restart and the filter-state snapshot (first and last box; the filter-state snapshot section), the broadcast-baseline jump response (`reactToEphemerisJumps_`, in the reaction box; the jump-response section) and the ambiguity-level check (`checkAmbiguityLevel_`; the ambiguity-level section).

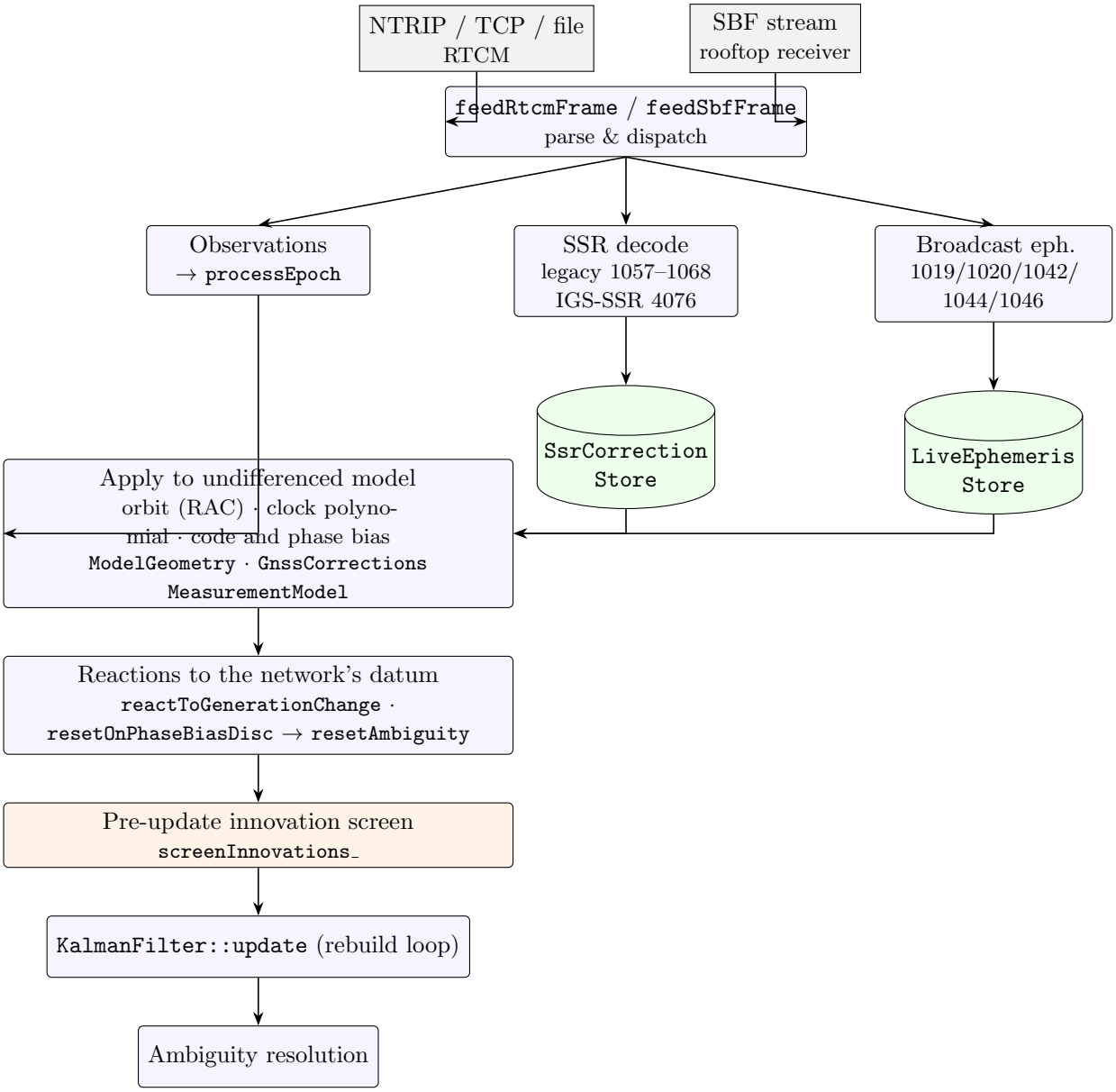


Figure 5: Real-time SSR correction pipeline. The decoder splits the byte stream three ways: observations go to `processEpoch`, SSR corrections to `SsrCorrectionStore`, and broadcast ephemerides to `LiveEphemerisStore`. The legacy SSR decoder accepts message types 1057–1068 and 1240–1270; the diagram lists the first range only. The undifferenced observation model then applies the precise orbit (RAC frame), the clock polynomial and the code and phase biases. After the slip check and before the Kalman update, `reactToGenerationChange` resets every network-corrected ambiguity when the product's solution generation changes, and `resetOnPhaseBiasDisc` answers a change of a phase-bias discontinuity counter (by default with an ambiguity reset), a hold generation, or a freshness flip of the applied bias; `reinflateOnCodeBiasDisc` and `reinflateOnClockDisc` follow for the code-bias and clock counters (the user's answer to the emitted product; the phase-bias step section of the SSR document). The innovation screen runs inside the update step ahead of the first pass. See `RealTimeEngine::processEpoch`.

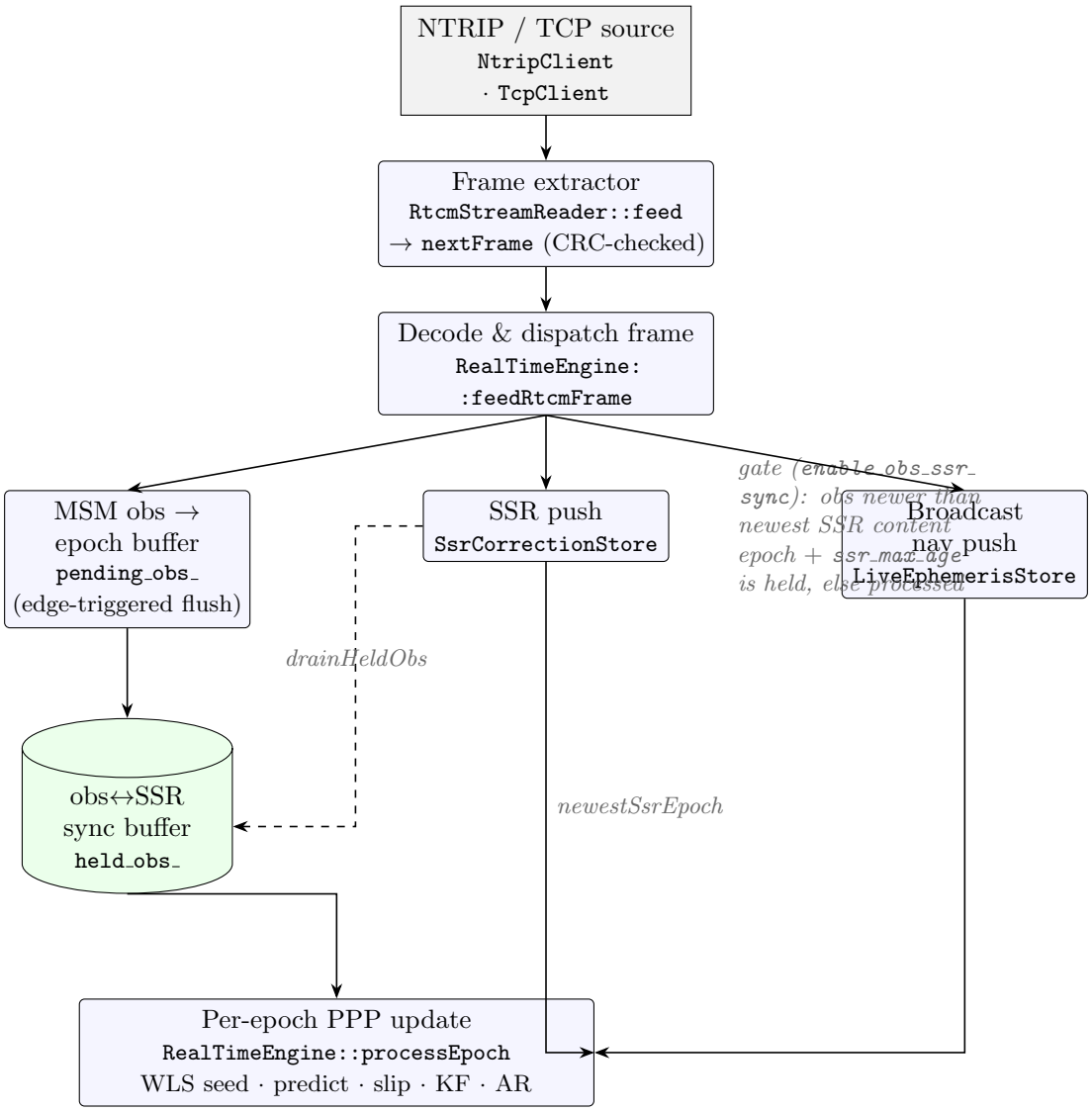


Figure 6: Real-time streaming front-end ahead of the SSR correction stage of Figure 5. `RtcmStreamReader` reassembles the raw bytes from `NtripClient` or `TcpClient` into CRC-validated RTCM3 frames, and `feedRtcmFrame` decodes and dispatches each frame. MSM observations accumulate in `pending_obs_`, which is submitted to `processEpoch` when a frame with a new epoch time arrives (the edge trigger). SSR corrections and broadcast ephemerides go to their stores. With `enable_obs_ssr_sync` set, the first admission gate of `processEpoch` (`admitBacklogAndSync`) holds an observation epoch that is newer than the SSR coverage horizon (newest SSR content epoch + `ssr_max_age`). The held epoch is kept in `held_obs_` (only the latest one) and resubmitted by `drainHeldObs` after each SSR push. An epoch inside the horizon is processed at once. A hold longer than `obs_ssr_sync_max_hold_s` (default 30 s) latches a degraded outage state in which epochs pass through without SSR until the stream covers them again. The gate applies only with a live SSR orbit and clock source and never before the first epoch. See the real-time engine.

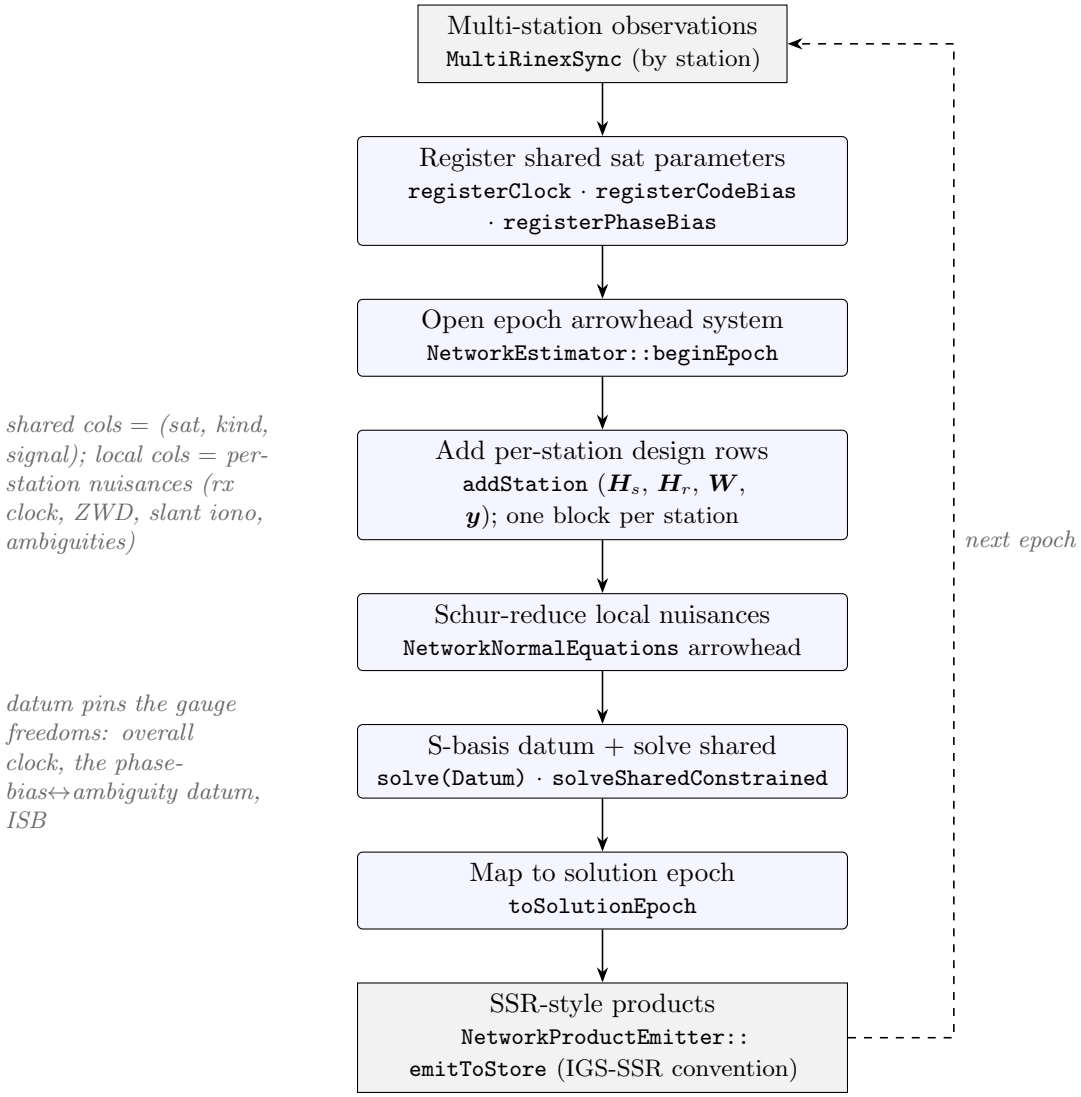


Figure 7: Joint estimator of the PPP-RTK network (`NetworkEstimator`, `network/solve/Estimator`). The shared satellite-side columns, one per (satellite, parameter kind, signal) for the estimable clock, code bias and phase bias, are registered afresh every epoch (`reset`, `registerClock`, `registerCodeBias`, `registerPhaseBias`; `registerVtecCoeff` serves the spherical-harmonic ionosphere, which the reference configurations do not use); `beginEpoch` opens an arrowhead normal-equation system of that width. `addStation` adds one pre-formed block of local-versus-shared design rows per station, with no receiver-side differencing. Schur reduction removes the local nuisances; the reduced shared system is rank-deficient by the gauge freedoms. `solve` fixes them with an explicit minimum-constraint S-basis datum on a copy of the reduced system, and `toSolutionEpoch` maps the estimate to a record that `NetworkProductEmitter` writes in the IGS-SSR convention. The driver only orchestrates: Figure 9 shows where the columns are registered and the rows added, Figure 11 the stages around the datum and the solve, Figure 12 the emission. With `network.closed_loop.enable` set, the emit step also drives a test station's PPP-RTK user on the new products: undifferenced in the reference configurations (`closed_loop.consumer_undifferenced`), between-satellite differenced otherwise. See the network mode and its architecture section.

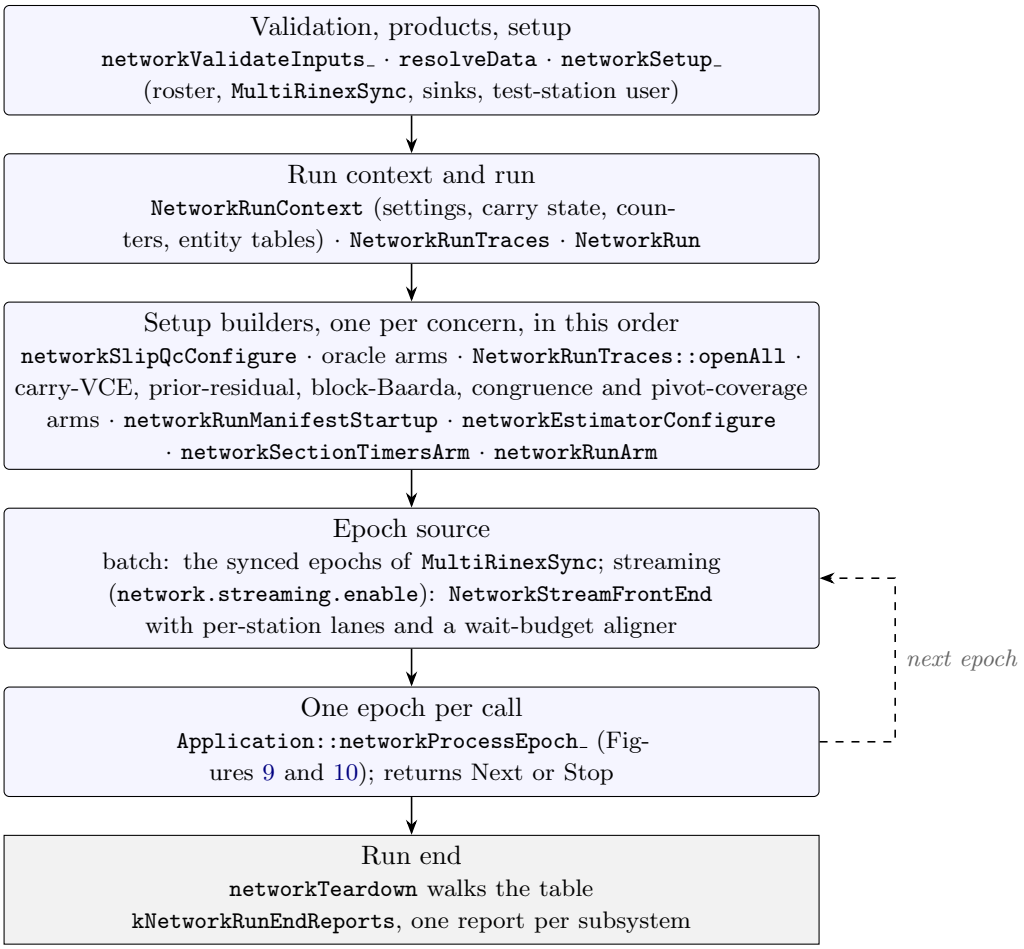


Figure 8: Structure of `Application::runNetwork`, the thin orchestrator of a network run (`network/driver/ApplicationNetwork`). Setup, the builders and the run end are lists of calls; the epoch loop has one body, whichever epoch source feeds it. A closed-loop replay of an existing product file replaces the whole run (`runConsumerReplay`) before validation.

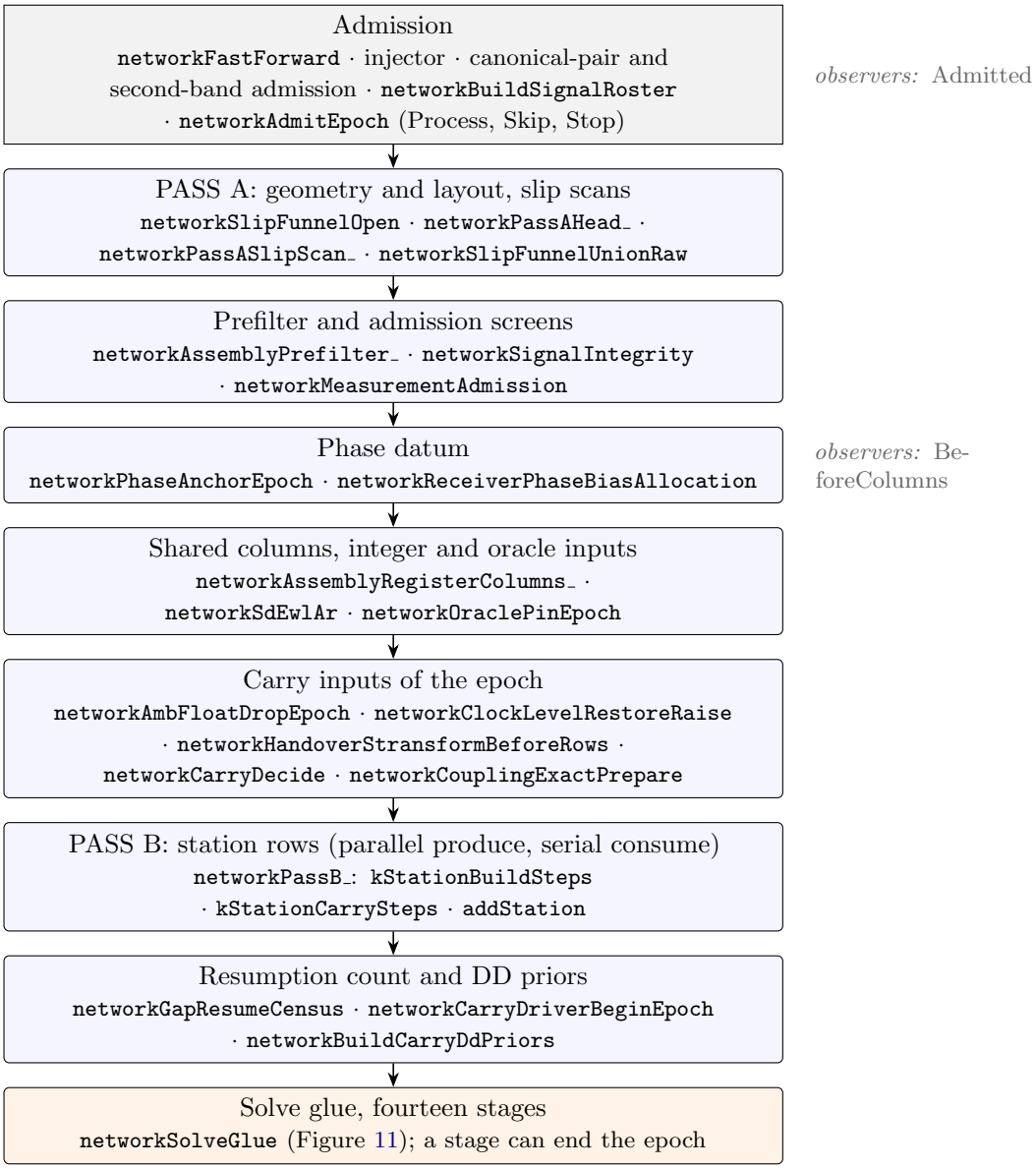


Figure 9: The epoch of `Application::networkProcessEpoch_` up to the solve. Each box is a seam of the form (`NetworkRun&`, `NetworkEpoch`) that owns no state; the epoch's values are grouped by the stage that writes them. The notes mark the two observer slots of this half. An observer is a measure-only instrument: it writes only its own counters and traces, and the table `kNetworkEpochObservers` (`network/driver/EpochObservers`) lists every observer with its slot and the reason for its place. Figure 10 continues the sequence.

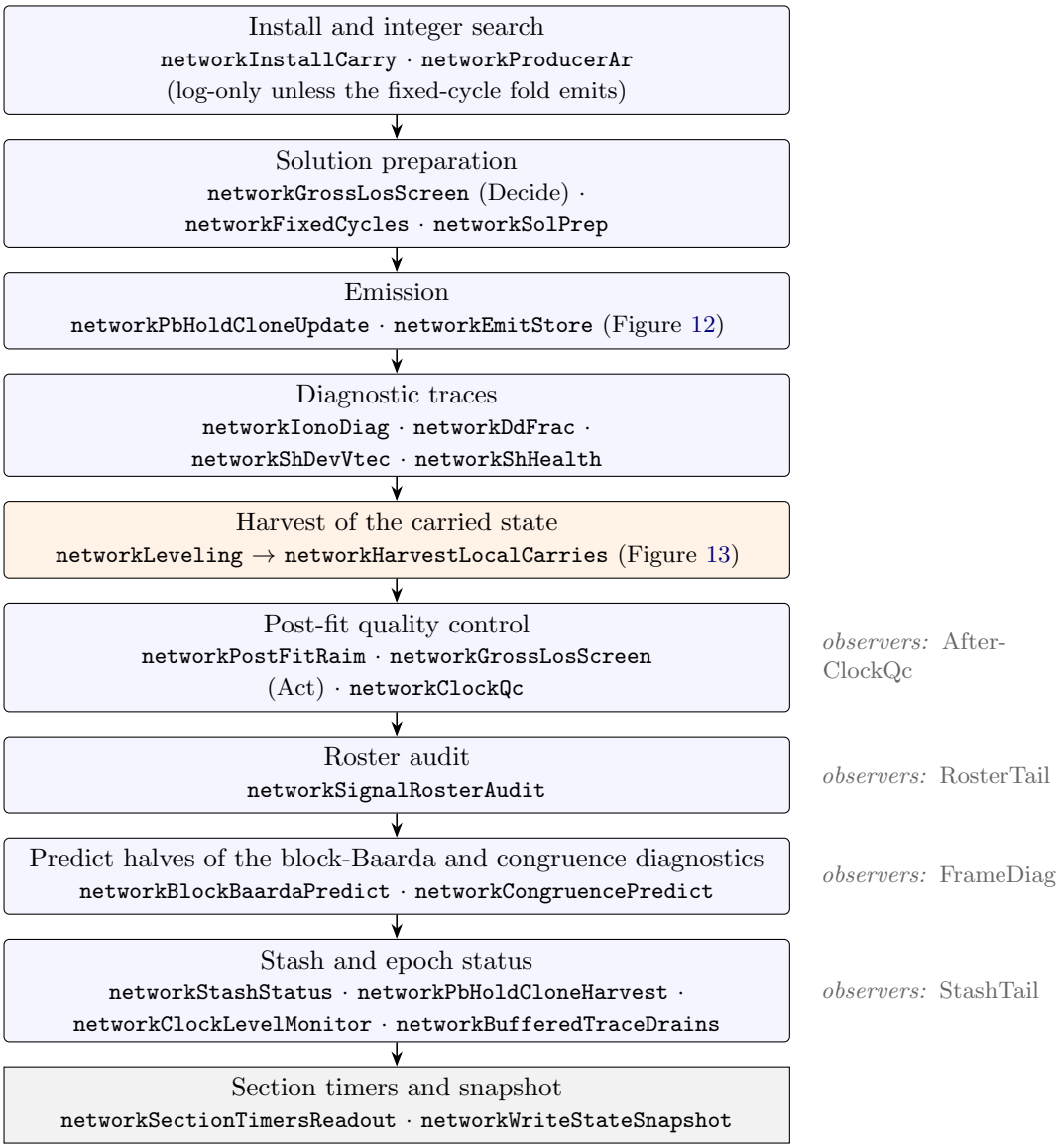


Figure 10: The epoch after the solve. The emission precedes the harvest, and the post-fit tiers follow the harvest because an erase of a carried entry is the refloat: it must act on the entry the harvest wrote. The notes name the four remaining observer slots, so the six slots of the observer table are, in spine order, *Admitted*, *BeforeColumns*, *AfterClockQc*, *RosterTail*, *FrameDiag* and *StashTail*. After the solve the only epoch values a seam writes are the hand-off to the post-fit tiers, the prepared solution and the epoch's events.

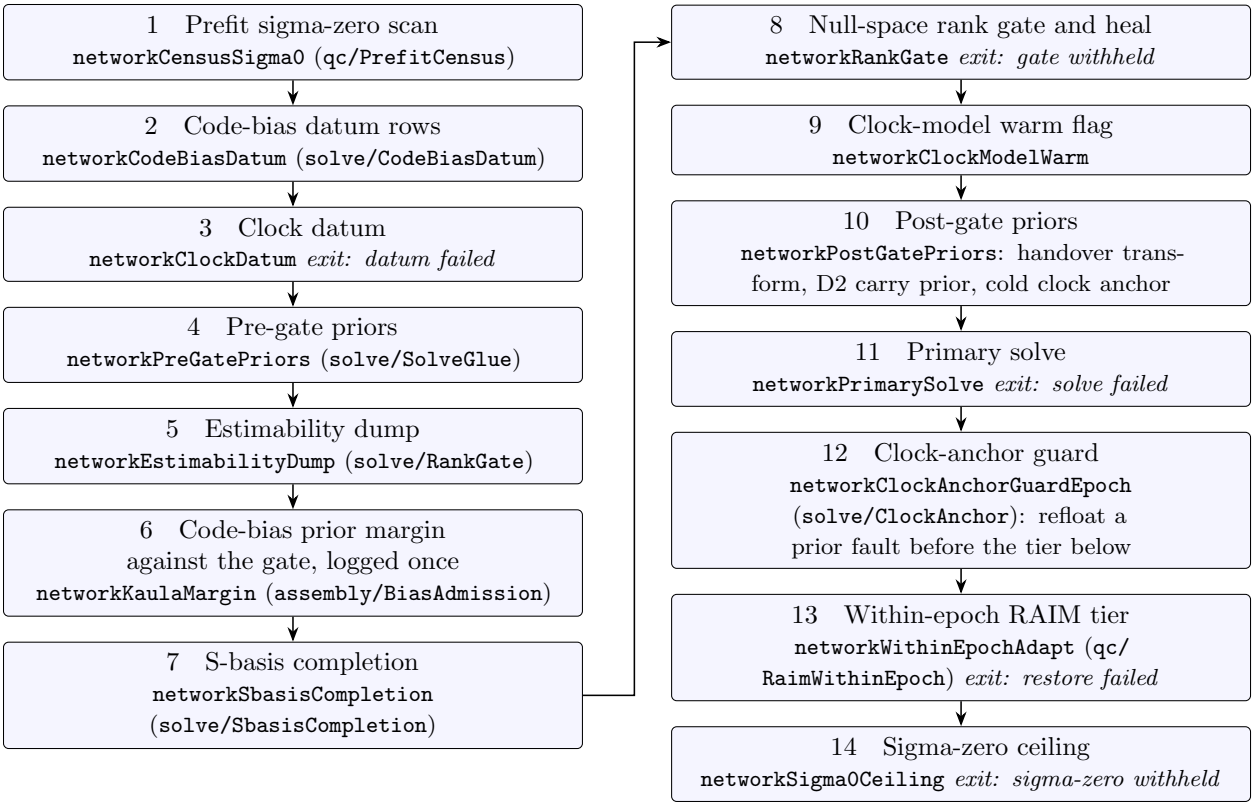


Figure 11: The fourteen stages of `networkSolveGlue` (`network/solve/SolveGlue`), in the order the estimator folds them: the order is load-bearing because the estimator accumulates the folds in this order and each stage reads what an earlier one wrote. Everything the rank gate must see is folded before stage 8; the gate, the dump and the heal read one tolerance. Stage 12 follows the solve and precedes stage 13, so that a prior fault is refloated before the tier can attribute it to data. Four stages can end the epoch, five ways, each a row of `kSolveSkips`. A new fold goes into stage 4 or stage 10 at its place in the order; a new way to end the epoch is a new row of the table and never an inline return. Unlabelled files are in `solve/`.

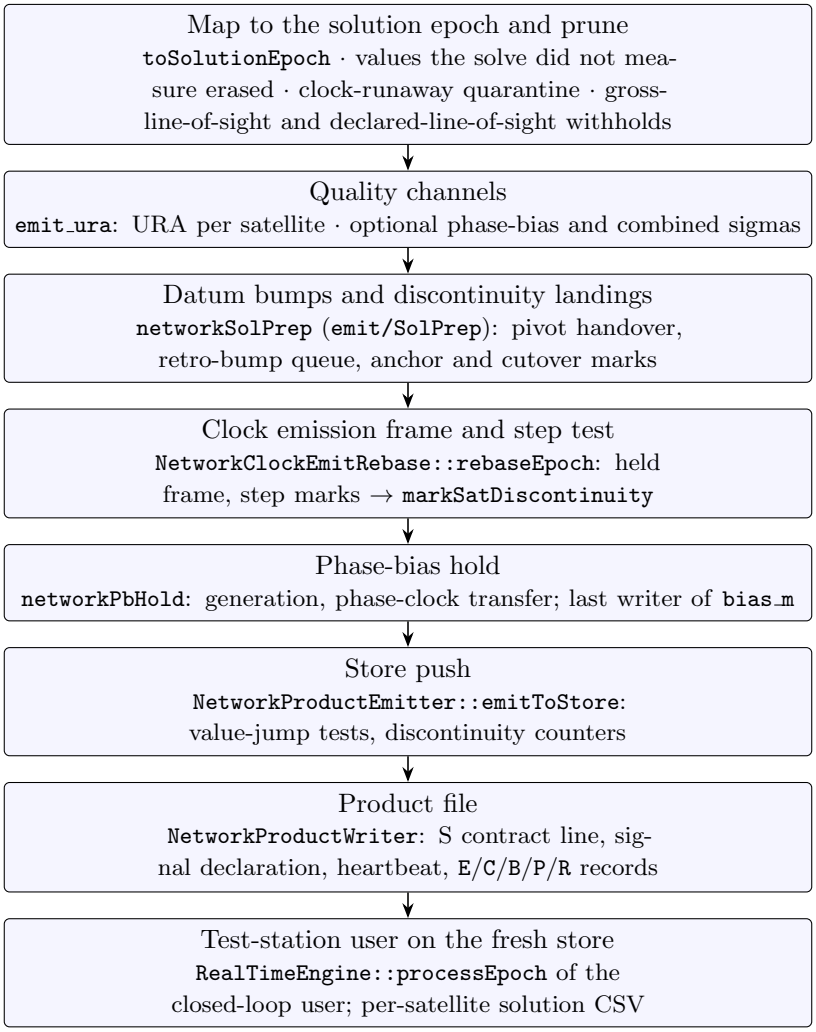


Figure 12: Emission, from the solved shared estimate to the product and the test station. The first three boxes are `networkSolPrep`, the last five `networkEmitStore` (the emit-layer frame re-base of the invariant carry is omitted, inactive in the reference configurations). Every step that changes an emitted value acts before `emitToStore` and before the file write, so the in-process store and a replay of the file carry the same values and reproduce the same discontinuity counters from the serialised records. The solve is unchanged by any step of this figure (the event-response and emitted-product section).

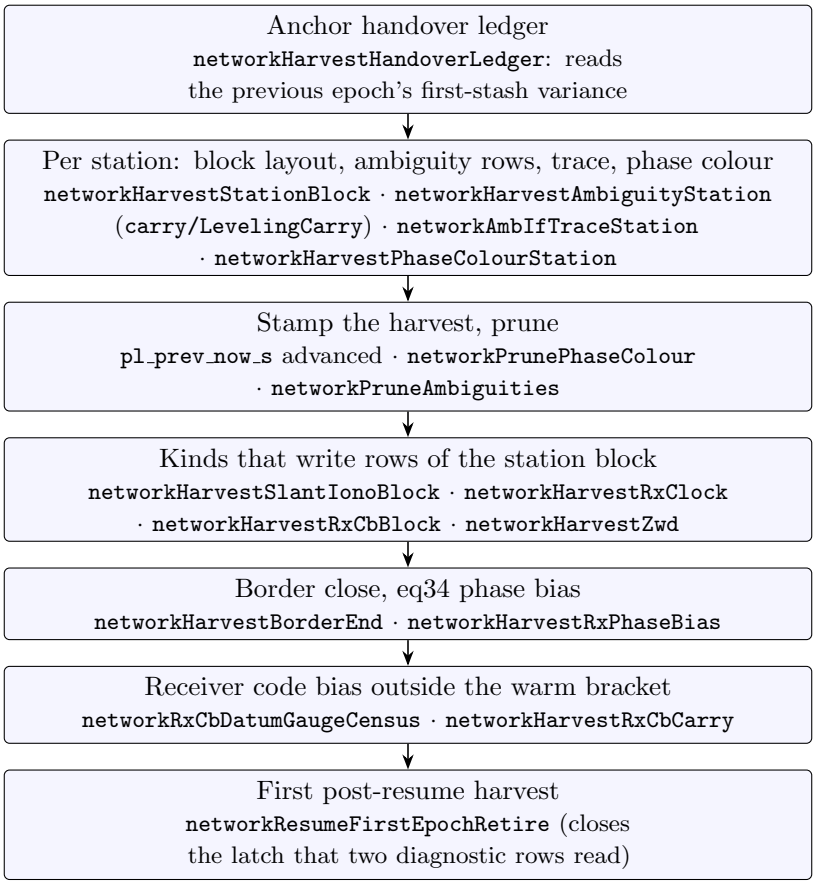


Figure 13: The post-solve harvest of the station-local carries, `networkHarvestLocalCarries` (`network/carry/LocalCarryHarvest`), called by `networkLeveling`. The orchestrator is a list of calls, one per carried kind, each in the file of its carry; the boxes from the second to the fifth run only with the carry warm. The order is fixed. The handover ledger reads the previous epoch's variance before the ambiguity stash rewrites it. Per station the layout, the ambiguity rows, the ionosphere-free trace and the phase colour come first, and the shared-history compression is sensitive to that order. The harvest clock advances after every station and before the prunes. Every writer of a row of the station block runs before the border is closed. The receiver code-bias carry runs outside the warm bracket because it must work on the invariant-carry lineage without phase leveling. RAIM and the gross line-of-sight screen run after the harvest because they set new marks. A carried column's regression goes through `networkHarvestColumn` (the carried local column).